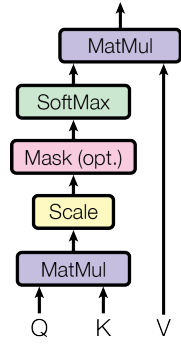


Scaled Dot-Product Attention



Multi-Head Attention

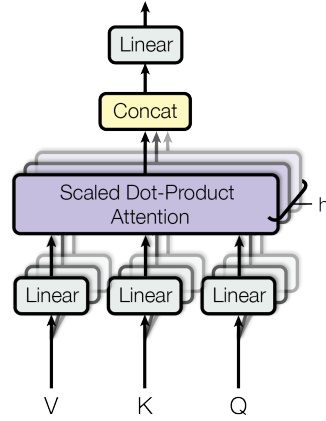


图 2: (左) 缩放的点积注意力。(右) 多头注意力由多个并行运行的注意力层组成。

值的权重，其中分配给每个值的权重是通过查询与相应键的兼容性函数计算的。

3.2.1 缩放点积注意力

我们将我们的特别注意力称为“缩放点积注意力”（图 2）。输入由维度 d_k 的查询和键以及维度 d_v 的值组成。我们使用所有键计算查询的点积，将每个键除以 $\sqrt{d_k}$ ，然后应用 softmax 函数来获取值的权重。

在实践中，我们同时计算一组查询的注意力函数，并将它们打包成一个矩阵 Q 。键和值也一起打包成矩阵 K 和 V 。我们将输出矩阵计算为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

两种最常用的注意力函数是加性注意力[2]和点积（乘法）注意力。除了 $\frac{1}{\sqrt{d_k}}$ 的缩放因子之外，点积注意力与我们的算法相同。附加注意力使用具有单个隐藏层的前馈网络来计算兼容性函数。虽然两者在理论复杂性上相似，但点积注意力在实践中更快、更节省空间，因为它可以使用高度优化的矩阵乘法代码来实现。

虽然对于较小的 d_k 值，两种机制的表现相似，但加性注意力优于点积注意力，而无需针对较大的 d_k 值进行缩放 [3]。我们怀疑，对于较大的 d_k 值，点积的幅度会变大，从而将 softmax 函数推入梯度极小的区域⁴。为了抵消这种影响，我们将点积缩放 $\frac{1}{\sqrt{d_k}}$ 。

3.2.2 多头注意力

我们发现，使用不同的学习线性投影分别将查询、键和值线性投影到 d_k 、 d_k 和 d_v 维度 h 次，而不是使用 d_{model} 维度的键、值和查询来执行单个注意力函数。On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional

⁴To illustrate why the dot products get large, assume that the components of q and k are independent random variables with mean 0 and variance 1. Then their dot product, $q \cdot k = \sum_{i=1}^{d_k} q_i k_i$, has mean 0 and variance d_k .